Ordinal Logistic Regression

</> Code

On this page

Ordinal Target Variable
Proportional Odds Model
Partial Proportional Odds Model
Interpretation
Predictions & Diagnostics

Ordinal Target Variable

Logistic regression is not limited to only have binary categorical target variables. Logistic regression works with target variables with any number of categories. If these categories have inherent order to them we perform **ordinal logistic regression**. Binary logistic regression is actually a special case of ordinal logistic regression since binary variables can only be written in two directions which is the requirement of ordinal variables. If the categories have no inherent order to them we perform **nominal (or multinomial) logistic regression**. Here we will discuss ordinal logistic regression.

Ordinal logistic regression models can also be used to model continuous target variables that have bounds on them however this will not be covered here.

Ordinal logistic regression models are generalizations of binary logistic regression models. In binary logistic regression we are calculating the probability that an observation has an event. In ordinal logistic regression we are calculating the probability that an observation has at most that event in an ordered list of outcomes.

We will be using the wallet data set to model the association between various factors and different levels of ethical responses to finding a wallet - returning the wallet and everything in it, returning the wallet but keeping the money found in it, keeping both the wallet and the money found inside. The variables in the data set are the following:

Variable	Description
male	indicator for a male student
business	indicator for a student enrolled at a business school
punish	how often the student was punished as a child (1 - low, 2 - moderate, 3 - high)
explain	indicator for whether explanation for punishment was given

The most common and easiest to interpret approach for ordinal logistic regression is the cumulative logit approach in the proportional odds model.

Proportional Odds Model

Instead of modeling the typical logit from binary logistic regression, we will model the cumulative logits. These cumulative logits are built off target variables with m categories. The first logit will summarize the first category compared to the rest. The second logit will summarize the first two categories compared to the rest. This continues $m-1$ times as the last logit compares the first $m-1$ categories to the m^{th} category. In essence, with a target variable with m categories, we are building $m-1$ logistic regressions. This is why binary logistic regression is a special case of the ordinal logistic regression model - 2 categories in the target variable leads to one logistic regression model.

The main assumption of the proportional odds model is that although the intercept changes across the $m-1$ models, the slope parameters on each variable stay the same. This will make the effects proportional across the different logistic regressions.

Let's see how to build these proportional odds models in each of our softwares!

R Python SAS

In R, we need to use the `polr` function instead of the `glm` function to perform ordinal logistic regression models. The formula piece of the function is much the same as with the `glm` function. The `method =` option specifies that we are performing a logistic regression while the `data =` option specifies the data frame being used.

▼ Code

```
train <- wallet
train$punish <- factor(train$punish)

library(MASS)

clogit.model <- polr(factor(wallet) ~ male + business + punish
                     method = "logistic", data = train)
summary(clogit.model)
```

```
Re-fitting to get Hessian

Call:
polr(formula = factor(wallet) ~ male + business + punish + expla
    data = train, method = "logistic")

Coefficients:
           Value Std. Error t value
male     -1.0598     0.3274  -3.237
business -0.7389     0.3556  -2.078
punish2  -0.6276     0.4048  -1.551
```

```
punish3  -1.4031     0.4823  -2.909
explain   1.0519     0.3408   3.086

Intercepts:
    Value   Std. Error t value
1|2 -2.5679  0.4190    -6.1287
2|3 -0.7890  0.3709    -2.1273

Residual Deviance: 307.3349
AIC: 321.3349
```

In the output above we see similar things to the output we have seen previously. R summarizes the model used as well as the parameter estimates of the model. The main difference in the parameter estimates is the multiple intercepts. Since we have three levels of our target variable we have two logistic regression models and therefore two intercepts.

The main question from here is whether the assumption of proportional odds actually holds. By default R doesn't provide a global test to answer this. However, the Brant test in the `brant` function will test each of the variables in the separate logistic regression models to see if their slope parameters are the same.

▼ Code

```r
library(brant)

brant(clogit.model)
```

```
--------------------------------------------
Test for     X2   df   probability
--------------------------------------------
Omnibus      5.46    5    0.36
male         0.51    1    0.47
business     0.58    1    0.45
punish2      0.99    1    0.32
punish3      2.81    1    0.09
explain      0.25    1    0.62
--------------------------------------------

H0: Parallel Regression Assumption holds
```

The null hypothesis for the above tests are that the assumption holds - the slopes parameter across all of the variables in the separate logistic regression models are equal (proportional odds). Since our p-values are above any reasonable significance level for this sample size, we have no evidence to say the assumption doesn't hold for any of the variables.

Partial Proportional Odds Model

Although our data doesn't have the proportional odds assumption fail in any of the variables, what would we do if it did? The partial proportional odds model is a model where some (not all) of the variables don't follow the proportional odds assumption. If none of the variables follow the proportional odds assumption, then we should build a nominal (or multinomial) logistic regression summarized in the next section.

Let's see how each of our softwares can build partial proportional odds models!

R Python SAS

R uses the `vglm` function to perform the partial proportional odds model. The formula piece of the function is the same as previous functions used as well as the `data =` option. The main difference is the `family =` option that specifies the cumulative logit. In the `cumulative` option we have the `parallel = F` option followed by which variables we think the assumption doesn't hold for - namely the *business* variable. This is just an example for completeness. According to the results from the Brant test above, **all** of the variables meet the assumption so we would not need the partial proportional odds model for this dataset.

▼ Code

```r
library(VGAM)

plogit.model <- vglm(factor(wallet) ~ male + business + punish
                     data = train, family = cumulative(parallel
summary(plogit.model)
```

```
Call:
vglm(formula = factor(wallet) ~ male + business + punish + expla
    family = cumulative(parallel = F ~ business), data = train)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept):1  -2.6695     0.4466  -5.978 2.26e-09 ***
(Intercept):2  -0.7730     0.3678  -2.102  0.03557 *
male            1.0707     0.3258   3.287  0.00101 **
business:1      0.9722     0.4789   2.030  0.04236 *
business:2      0.6376     0.3810   1.674  0.09423 .
punish2         0.6300     0.4008   1.572  0.11594
punish3         1.3956     0.4727   2.952  0.00316 **
explain        -1.0532     0.3413  -3.086  0.00203 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Names of linear predictors: logitlink(P[Y<=1]), logitlink(P[Y<=2

Residual deviance: 306.8392 on 382 degrees of freedom

Log-likelihood: -153.4196 on 382 degrees of freedom

Number of Fisher scoring iterations: 5

No Hauck-Donner effect found in any of the estimates


Exponentiated coefficients:
      male business:1 business:2    punish2    punish3    explai
 2.9174165  2.6438481  1.8918696  1.8776731  4.0373205  0.348807
```

The above output is the same as the proportional odds model output with one difference. There are two slope parameter estimates for the *business* variable - one for each model. The rest of the output is the same as before.

Interpretation

To understand the interpretation of the odds ratios in ordinal logistic regression we need to remember how the logistic regression equations are structured. Instead of modeling the typical logit from binary logistic regression, we will model the cumulative logits. These cumulative logits are built off target variables with m categories. The first logit will summarize the first category compared to the rest. The second logit will summarize the first two categories compared to the rest. This continues $m-1$ times as the last logit compares the first $m-1$ categories to the m^{th} category. The question really becomes what is first. Is it the largest category or the smallest category? The answer to this question will influence the interpretation of the results. If the first category is the smallest category then the categories are ascending as they build up further logistic regression models. If the first category is the largest category then the categories are descending as they build up further logistic regression models.

Both SAS and R can easily calculate descending category ordinal logistic regressions. The highest valued category is the starting point and each additional logistic regression adds the next highest valued category - building from the top down.

R Python SAS

By default, the `polr` function in R builds ordinal logistic regression models in descending order of categories. To get the odds ratios from R we just exponentiate the parameter estimates from the logistic regression model.

▼ Code

```
ORtable <- data.frame(OR = exp(coef(clogit.model)),
                      lower = exp(confint(clogit.model))[,1],
                      upper = exp(confint(clogit.model))[,2])
print(ORtable)
```

```
                OR      lower     upper
male     0.3465172 0.17995548 0.6522567
business 0.4776512 0.23775967 0.9626137
punish2  0.5338490 0.24246863 1.1934058
punish3  0.2458364 0.09418942 0.6315739
explain  2.8630214 1.46945567 5.6121973
```

Let's see how the odds ratios are now interpreted with the descending order of categories. Odds ratios in descending ordinal logistic regressions are interpreted as having higher odds of being in a **higher** category for the target variable. For the *male* variable, males are 0.347 times more likely to have a higher value on our ethical scale as compared to non-males. A better way to think of this is that on average, males have 65.3% lower odds of being in a higher ethical scale. Let's remember how our scale is calculated. A value of 1 keeps the wallet and the money, 2 keeps the money but returns the wallet, and 3 returns both the money and the wallet. For the *business* variable, business school students have on average, 52.2% lower odds of being in a higher ethical category as compared to non business school students.

Predictions & Diagnostics

Ordinal logistic regression has a lot of similarities to binary logistic regression:

- Multicollinearity still exists
- Non-convergence problems still exists
- Concordance, Discordance, Tied pairs still exist and so does the c statistic
- Generalized R^2 remains the same

There are some inherent differences though between binary and ordinal logistic regression. A lot of the diagnostics cannot be calculated for ordinal logistic regression. ROC curves and residuals cannot typically be calculated because there is actually more than one logistic regression occurring.

Predicted probabilities are actually for each category though. Let's see how we can get predicted probabilities from each of our softwares!

R Python SAS

In R we have the same process of getting predicted probabilities in ordinal logistic regression as we did for binary logistic regression. The `predict` function will score any data set. The `type = probs` option is used to get individual predicted probabilities for each category.

▼ Code

```
pred_probs <- predict(clogit.model, newdata = train, type = "pr
head(pred_probs)
```

```
           1         2         3
1 0.12562481 0.3341195 0.5402557
2 0.04778463 0.1813420 0.7708734
3 0.02609095 0.1108549 0.8630542
4 0.12562481 0.3341195 0.5402557
5 0.07176375 0.2423258 0.6859105
6 0.02609095 0.1108549 0.8630542
```

The three variables are the individual predicted probabilities for each of the categories of our target variable. Unlike binary logistic regression where we typically did not use a cut-off of 0.5 to determine which category we predict, in ordinal logistic regression we typically pick the category with the highest probability as our predicted category.

Accuracy is typically not as high in logistic regressions with more than two categories because there are more chances for incorrect predictions. However, people do rank errors in ordinal logistic regression. For example, if the actual ethic score was 1, then a prediction of 2 is better than a prediction of 3.

← Model Assessment

Multinomial Logistic Regression →